

Python & Tools

Программирование, 2024-2025

Терещенко В.В.
TG: @VI-Tershch
Email: vlad-tershch@yandex.ru

Наше пространство

- Weeek: <https://app.weeek.net/ws/622674/document/11>
- Telegram Community: <https://t.me/+01QRG8GmwBQ5M2Zi>
- My Telegram - @VladTershch



Notion



Telegram Community

П2

Программирование 2024/2025

Шаблоны практических работ:

GitHub - VI-Tershch/cs-102-template

Contribute to VI-Tershch/cs-102-template development by creating an account on GitHub.

<https://github.com/VI-Tershch/cs-102-template>

VI-Tershch/cs-102-template

1 Contributor

0 Issues

0 Stars

0 Forks

Презентации лекций:

1 курс Программирование Лекции – Google Диск

Без описания

https://drive.google.com/drive/folders/1j7Fbv_HV3G9pMdnbOc7FCiWLBb-B43MN?usp=shari...

Ведомость с баллами:

1 Курс Программирование Ведомость

Без описания

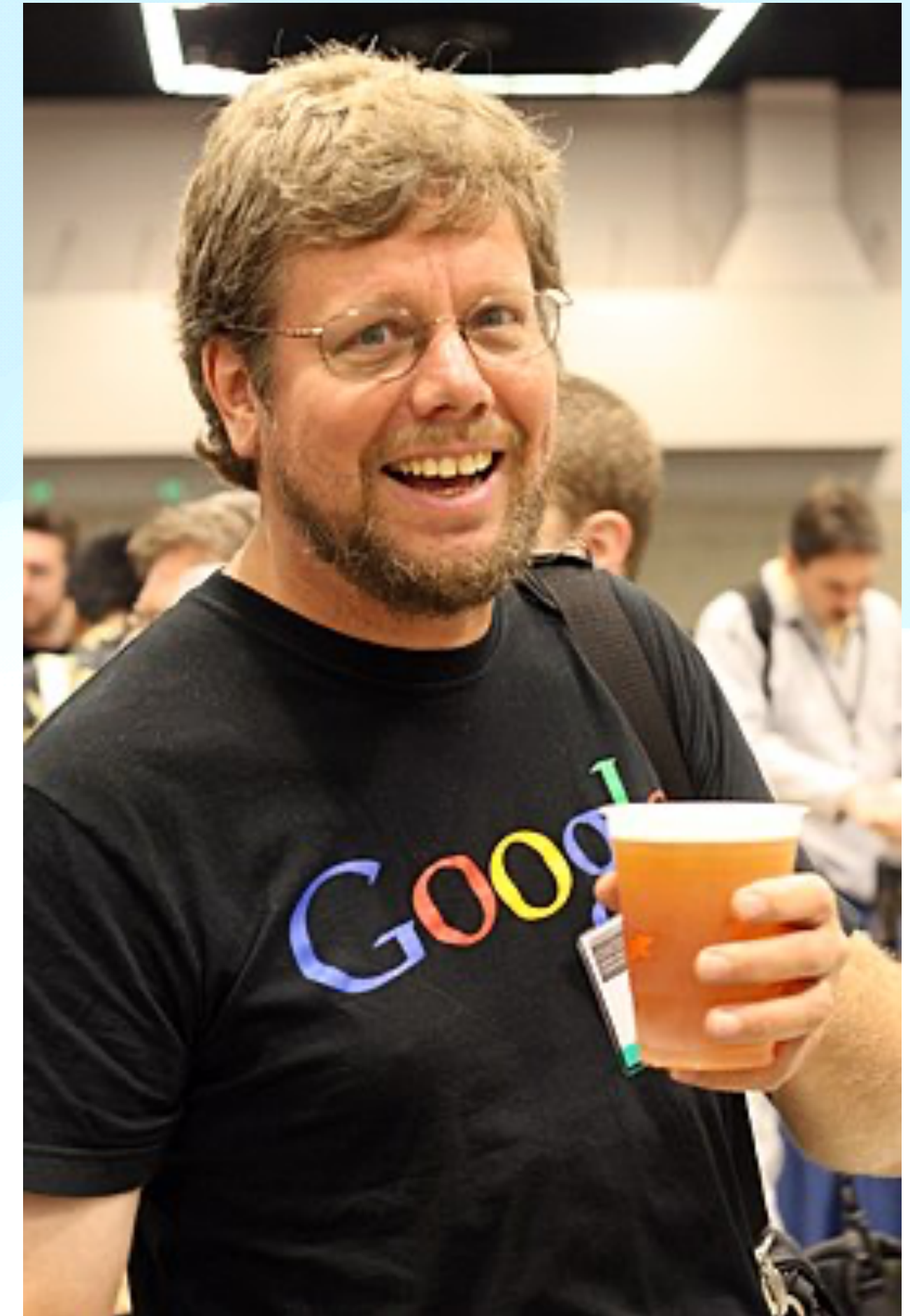
<https://docs.google.com/spreadsheets/d/1jxZ6gEdt5...>

Для отчетов: пишите на почту - ddiavall@yandex.ru

История

Гвидо ван Россум

- 1989 - попытка написать интерпретатор ради веселья;
- 1991 - релиз Python 0.9.0;
- 1994 - Python 1.0;
- 2000 - Python 2.0;
- 2008 - Python 3.0;
- С конца 2020 сообщество поддерживает только 3ю версию.



Где используется?

- Веб-разработка;
- Аналитика данных;
- Machine Learning;
- Автоматизация всего и вся.



Особенности языка

- Python - интерпретируемый язык;
- Динамическая типизация;
- Язык высокого уровня и простой в использовании;
- Соответствует принципам объектно-ориентированного программирования (ООП).

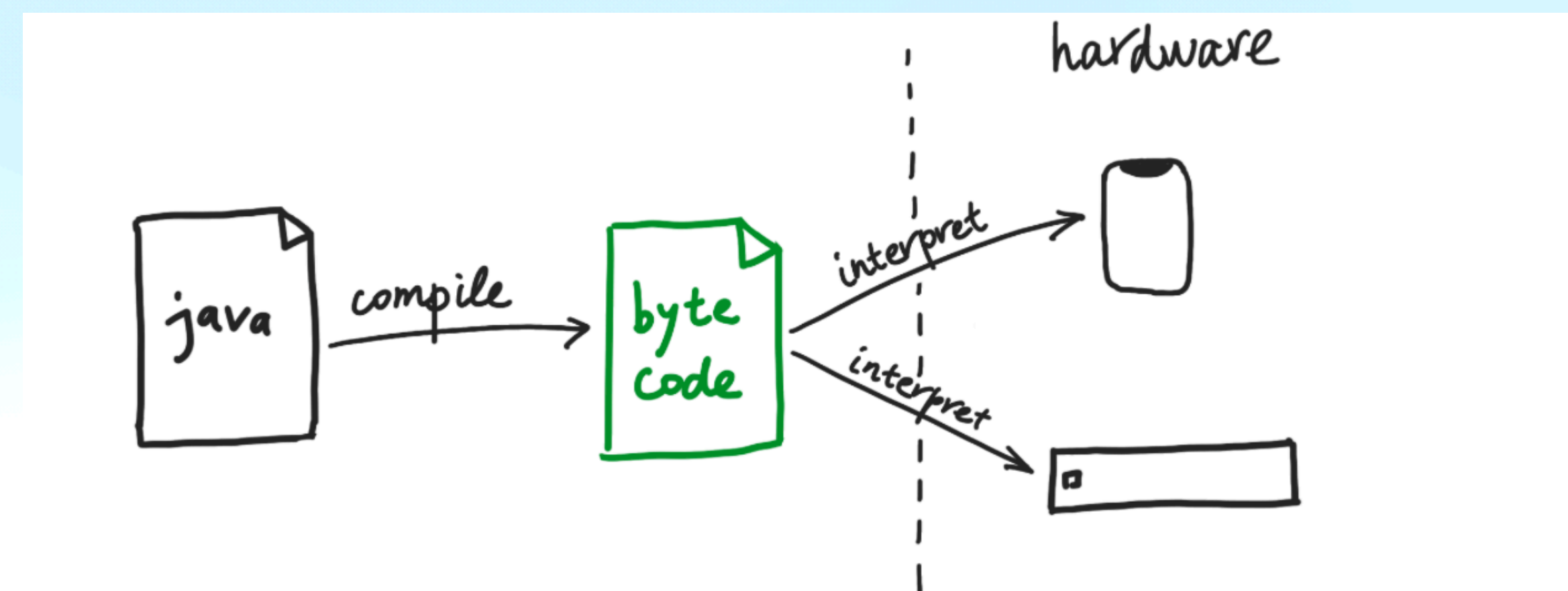
Компилируемые и интерпретируемые языки

Мы полагаемся на такие инструменты, как компиляция и интерпретация, чтобы преобразовать наш код в форму, понятную компьютеру. Код может быть исполнен нативно, в операционной системе после конвертации в машинный (путём компиляции) или же исполняться построчно другой программой, которая делает это вместо ОС (интерпретатор).

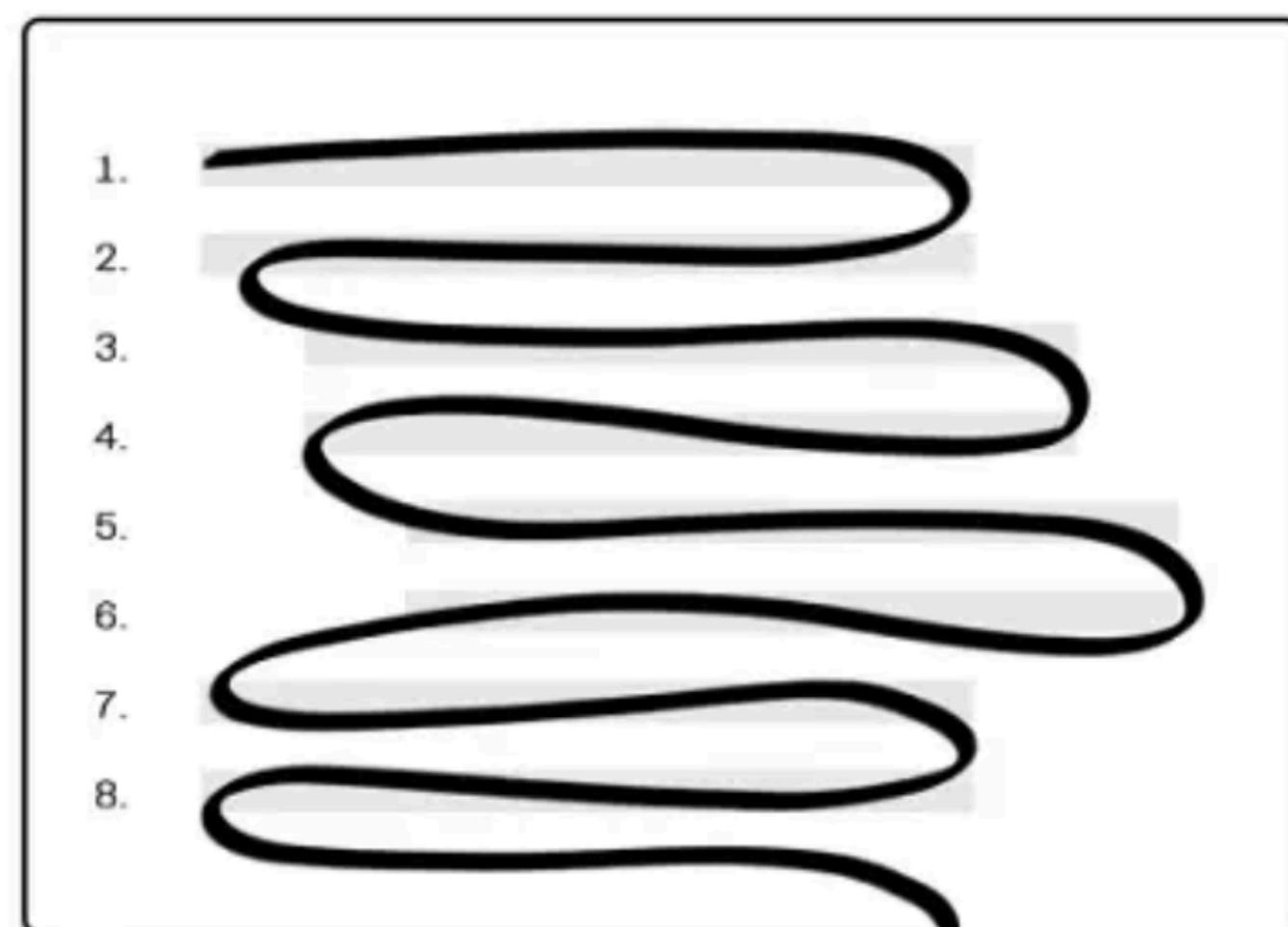
- Компилируемый язык — это такой язык, что программа, будучи скомпилированной, содержит инструкции целевой машины; этот машинный код непонятен людям (C, C++, Haskell, Go).
- Интерпретируемый же язык — это такой, в котором инструкции не исполняются целевой машиной, а считываются и исполняются другой программой (которая обычно написана на языке целевой машины) (Python, PHP, Ruby).

Но есть и байткод-языки

В байткод-языке сперва происходит компиляция программы из человекочитаемого языка в байткод. Байткод — это набор инструкций, созданный для эффективного исполнения интерпретатором и состоящий из компактных числовых кодов, констант и ссылок на память. С этого момента байткод передаётся в виртуальную машину, которая затем интерпретирует код также, как и обычный интерпретатор.

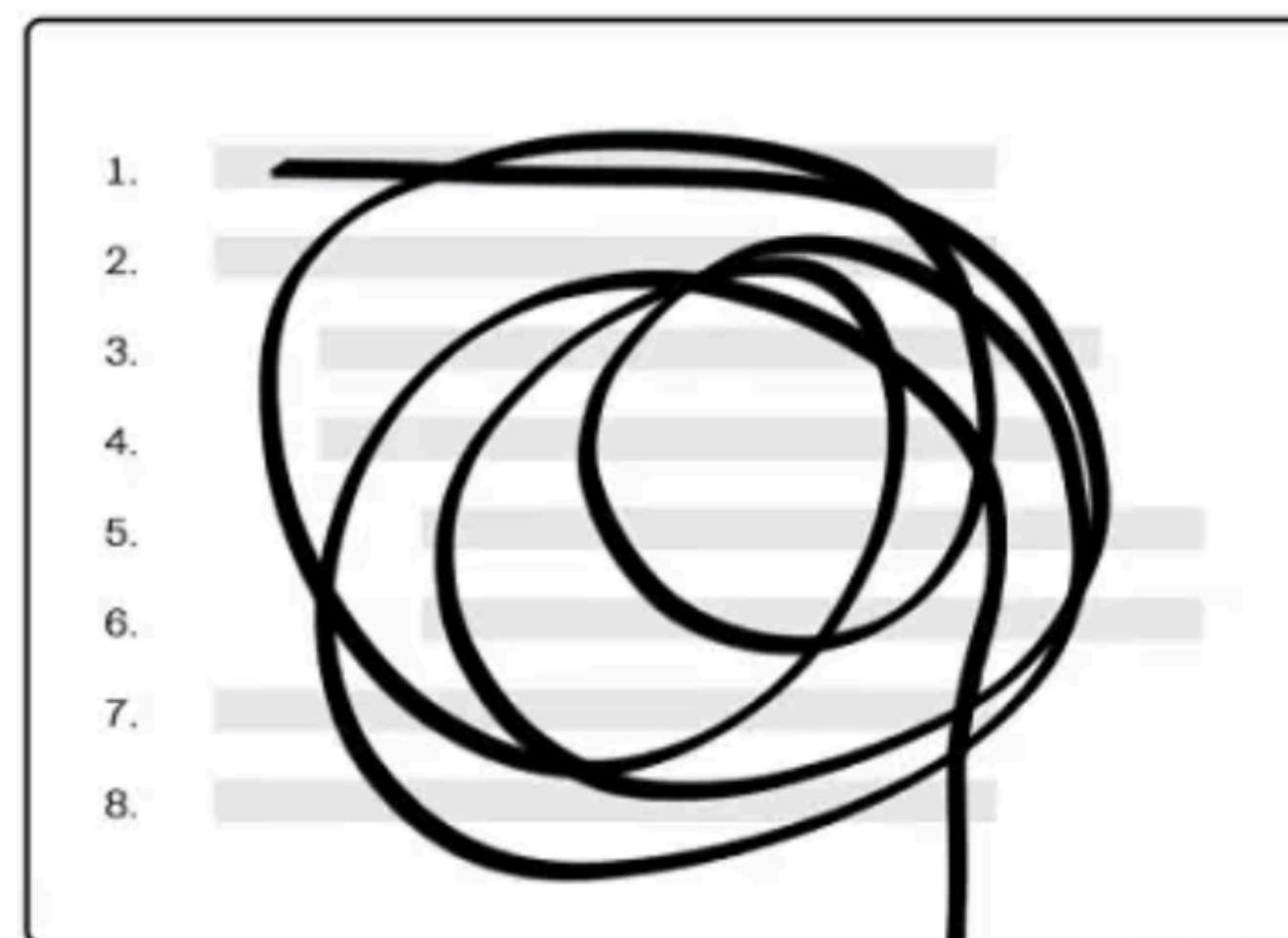


Интерпретируемый



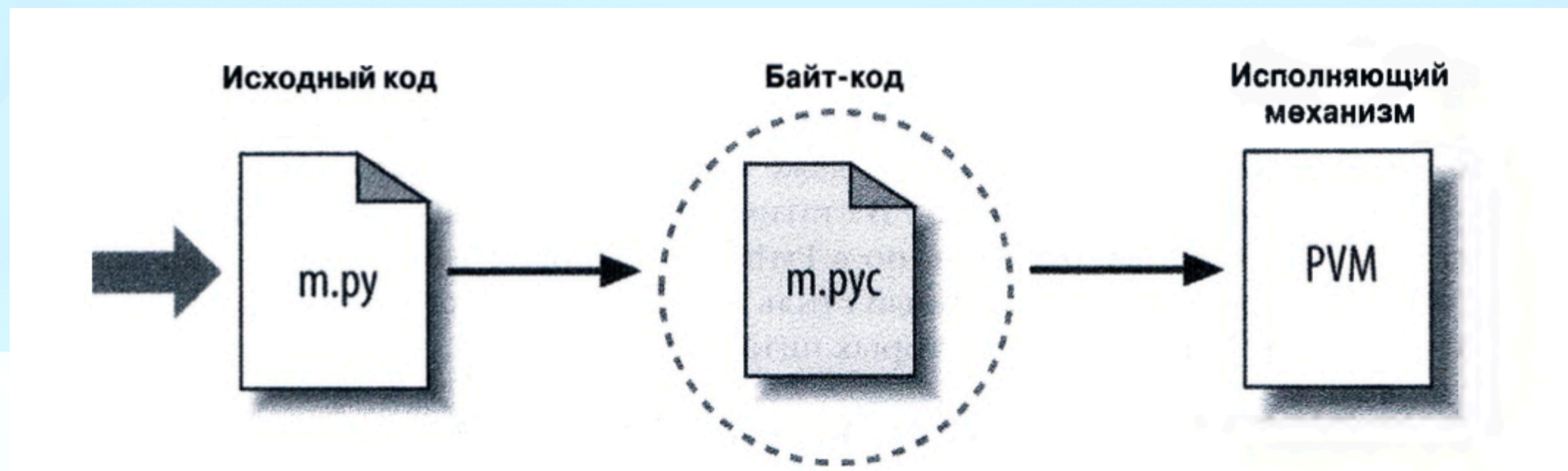
Hello, World!

Компилируемый



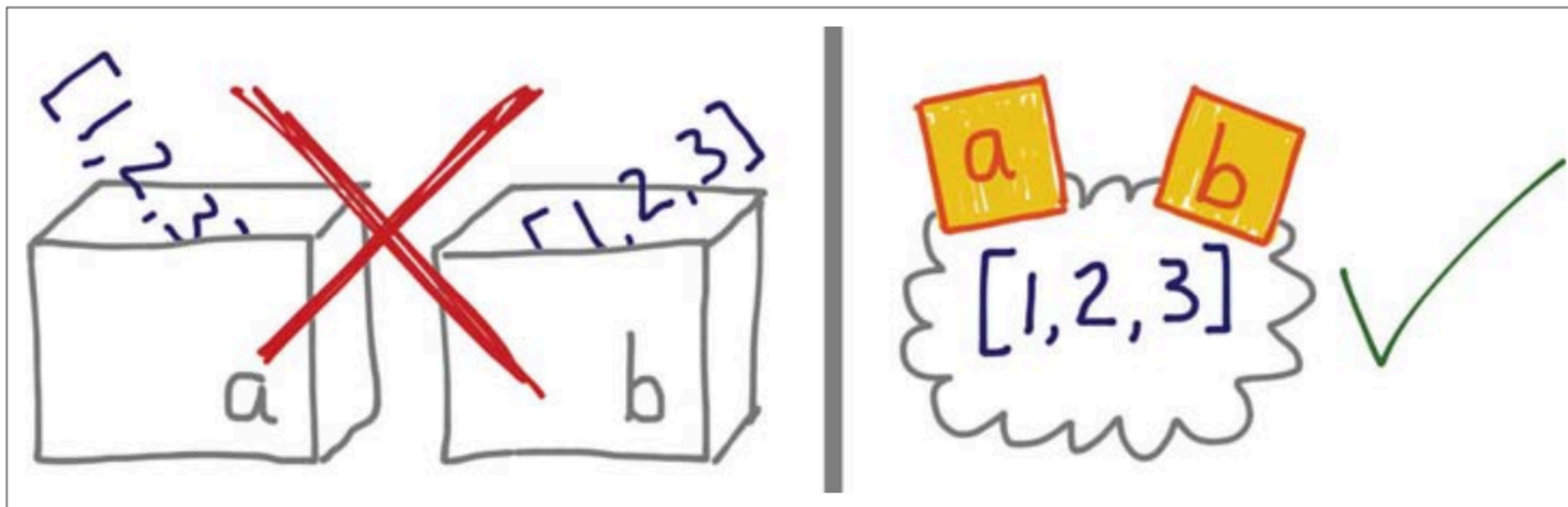
Hello, World!

CPython



- в работе Python обычно отсутствует шаг построения или “сборки”: код выполняется немедленно после того, как был набран.
- байт-код Python **НЕ** является двоичным машинным кодом!

Объекты



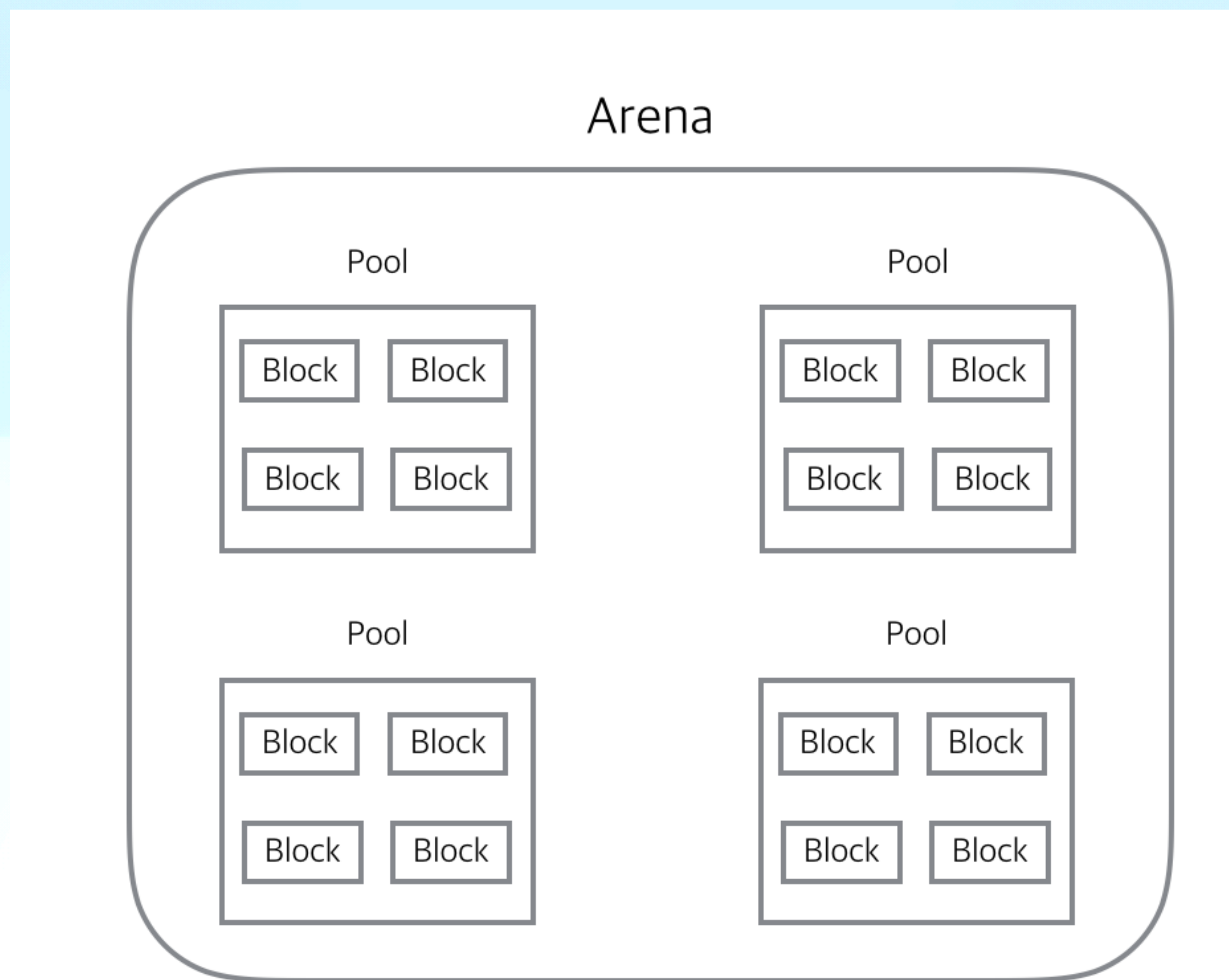
```
a = [1, 2, 3]
b = a
a.append(4)
print(b)
```

[1, 2, 3, 4]

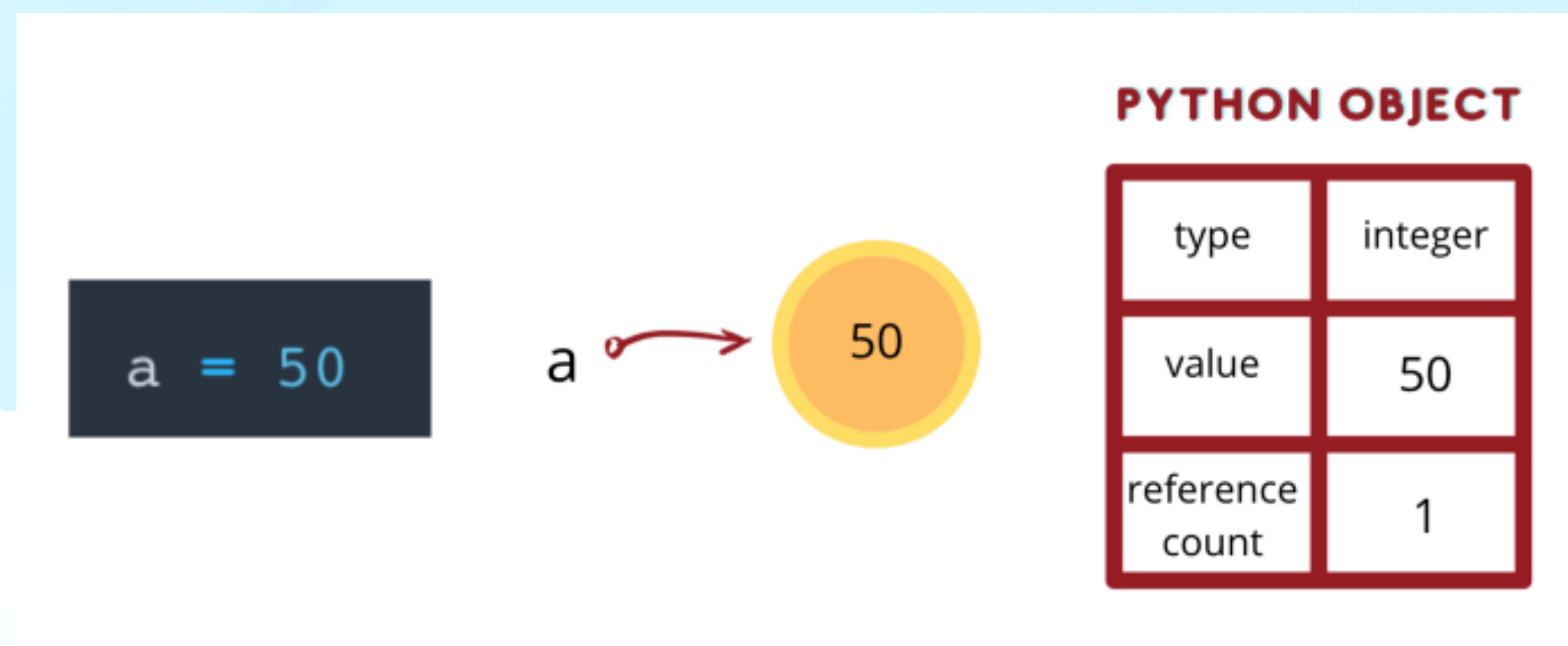
```
a = 4
b = a
a = a + 2
print(a, b)
```

6 4

Механизм памяти в Python



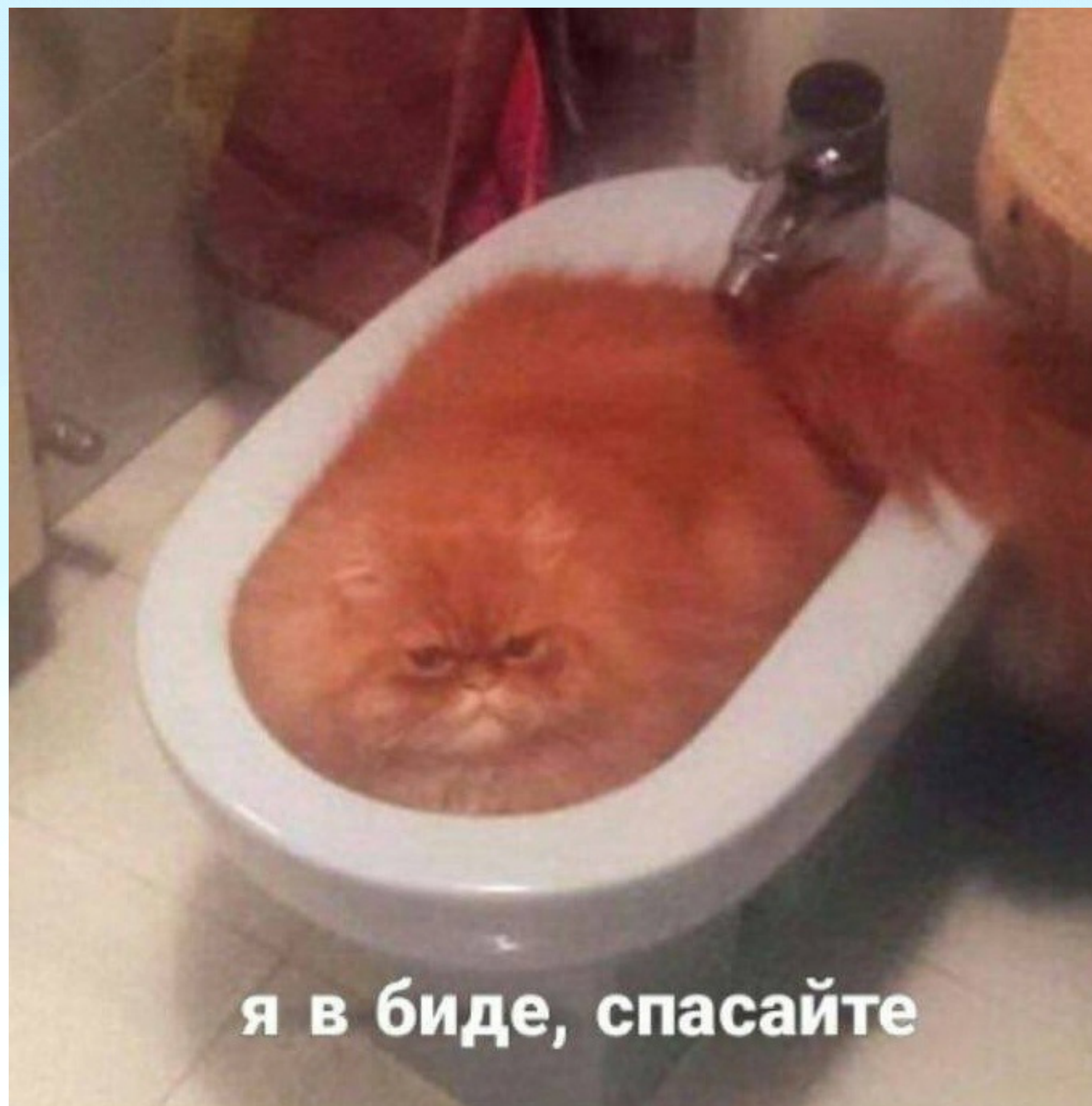
Жизнь объектов в памяти



- Счетчик ссылок
- Сборщик мусора

Типы объектов

Тип объекта	Пример литерала/создания
Числа	<code>1234, 3.1415, 3+4j, 0b111, Decimal(), Fraction()</code>
Строки	<code>'spam', "Bob's", b'a\x01c', u'sp\xc4m'</code>
Списки	<code>[1, [2, 'three'], 4.5], list(range(10))</code>
Словари	<code>{'food': 'spam', 'taste': 'yum'}, dict(hours=10)</code>
Кортежи	<code>(1, 'spam', 4, 'U'), tuple('spam'), namedtuple</code>
Файлы	<code>open('eggs.txt'), open(r'C:\ham.bin', 'wb')</code>
Множества	<code>set('abc'), {'a', 'b', 'c'}</code>
Прочие основные типы	Булевские значения, сами типы, <code>None</code>
Типы программных единиц	Функции, модули, классы
Типы, связанные с реализацией	Скомпилированный код, трассировки стека



я в биде, спасайте

Несколько общих рекомендаций

- Адекватный профиль в ТГ (хотя-бы имя/фамилия 🥰)
- Читайте чат(ы), пожалуйста 🥰🥰
- С преподавателями на «вы» и пишите кто вы 🥰🥰🥰

Система контроля версий

Управление исходным кодом (ака Контроль версий) — это практика отслеживания изменений программного кода и управления ими.

- Пишем осмысленные сообщения к коммитам/PR;
- Решение каждой задачи в отдельной ветке;
- Соблюдаем общепринятые конвенции - <https://www.conventionalcommits.org/en/v1.0.0/>



Преимущества систем контроля версий

- повышение скорости разработки, обеспечивая эффективную кооперацию;
- отслеживание каждого изменения (кто, когда, по какой причине);
- снижение вероятности ошибок и конфликтов при разработке за счет отслеживания каждого небольшого изменения;
- для каждого отдельного участника проекта поддерживается отдельная рабочая копия;
- оперативное восстановление в случае ошибок или непредвиденных ситуаций.

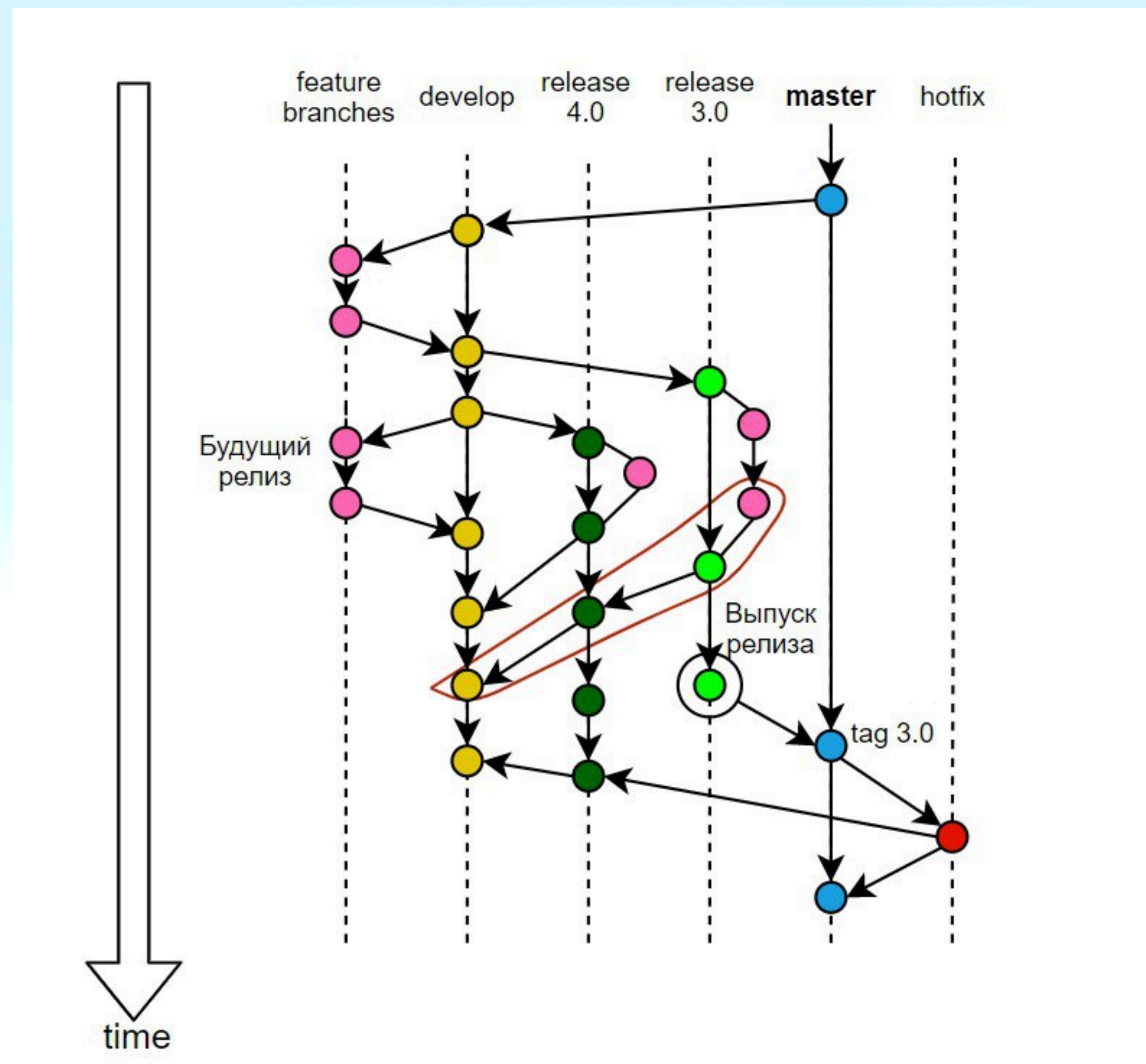
Примеры VCS



Git

Основные понятия:

- Ветки (бранчи);
- Коммиты;
- Pull Request / Merge Request



Git

Сервисы для работы с Git:

- GitHub
- GitLab
- Bitbucket (изначально был Mercurial Repo)

Способы работы:

- Консольная утилита
- GUI клиенты (SourceTree, Sublime Merge, Tortoise)
- Интегрированное в IDE решение

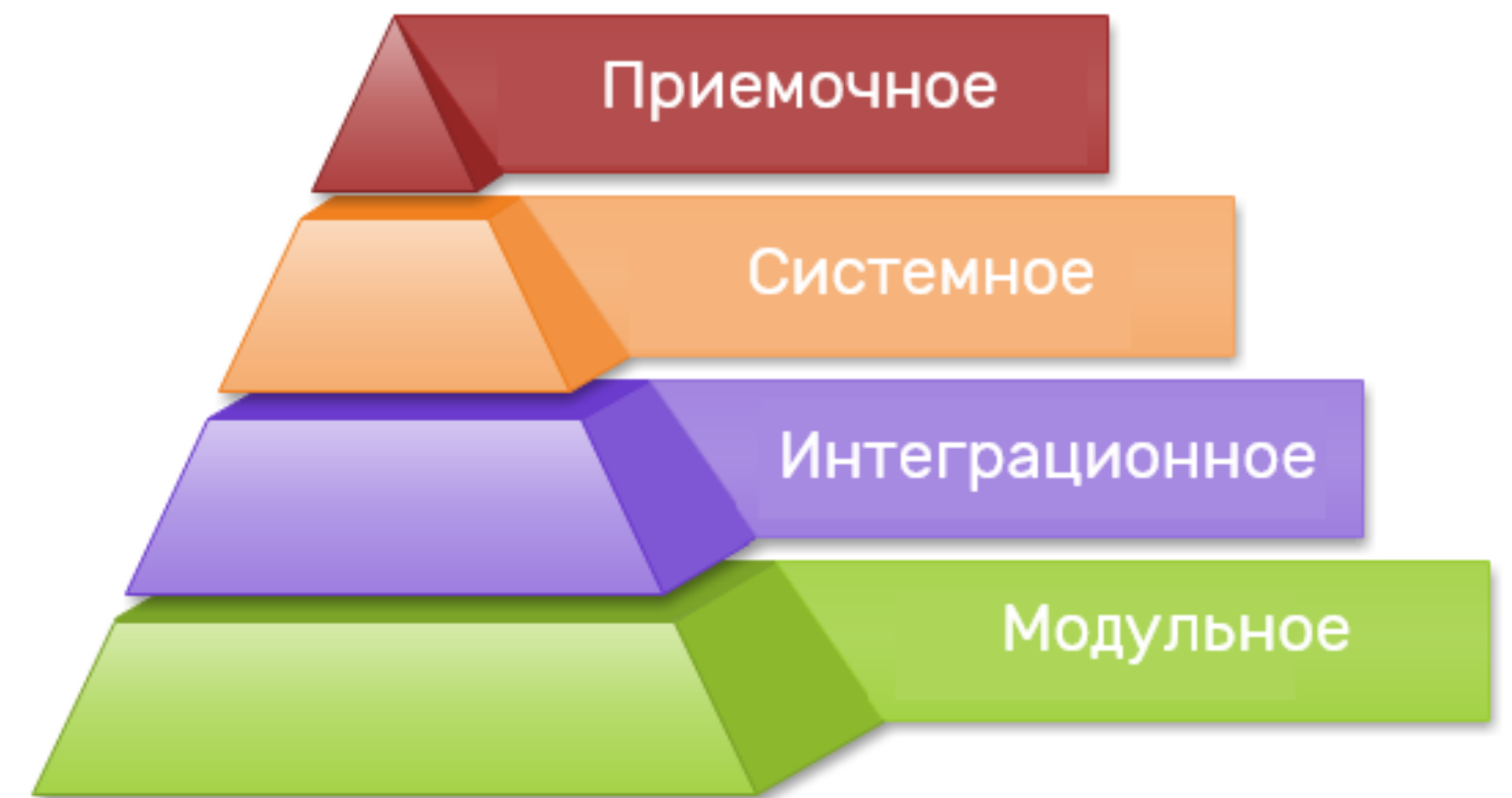
Обеспечение качества и тестирование

Quality assurance, QA - набор процедур мониторинга разработки программного обеспечения и методов, используемых для обеспечения его качества. Процедуры обеспечения качества охватывают весь цикл разработки программного обеспечения.

Тестирование программного обеспечения, QC — процесс исследования, испытания программного продукта, имеющий своей целью проверку соответствия между реальным поведением программы и ее ожидаемым поведением на конечном наборе тестов, выбранных определенным образом.

Классификация по подходу и по уровням

- Модульное (unit)
- Интеграционное (integration)
- Системное (e2e)
- Тестирование на пользователях



Артефакты

- Стратегия тестирования
- План тестирования
- Тест кейсы
- Автотесты
- Отчет о тестировании
- nextAction по результатам



Критерии хорошего теста

- **актуальность** - тест действительно проверяет то что должен
- **изолированность** - тест проверяет одну функцию объекта тестирования
- **независимость** - тест не зависит от состояния системы и других тестов
- **однозначность** - в тесте нет ветвления проверок поведения в зависимости от скрытых состояний системы
- **повторяемость** результата - при повторных прогонах тест выдает один и тот же результат
- **надежность** - тест падает только при выполнении условий Assert
- наличие понятного вывода результатов проверки

Тестирование позволяет объективно оценить насколько разработанный продукт соответствует требованиям!

Автоматизация тестирования

Автоматизированное тестирование - это любой вид тестирования, выполненный с применением программных инструментов.

Мы затронем три вида тестирования:

- функциональное компонентное тестирование
- интеграционное тестирование
- системное тестирование

Модульные тесты

Объект тестирования - класс, метод класса, функция

Цель теста - изолировать отдельные части программы и показать, что по отдельности эти части работают согласно функциональным требованиям.

Когда такие тесты не подойдут:

- Результат известен лишь приблизительно (сложные разветвленные алгоритмы, моделирование)
- процесс организовывается относительно кода, взаимодействующего с системой

Интеграционные тесты

Объект тестирования - программные модули, группы компонент

Цель тестирования - проверка корректности взаимодействия компонент согласно контрактам

Требуют развертывания большего окружения по сравнению с модульными.

При реализации часто возникает необходимость создавать заглушки и МОС сервисы для обеспечения возможности проверки модулей.

Системные тесты

Объект тестирования - система, продукт или большая обособленная часть системы

Цель - проверка соответствия системы полностью развернутой системы исходным требованиям.

Как правило системные тесты сложнее в разработке и дороже в поддержке, но именно такие тесты показывают что вся система работает как надо, и позволяют убедить в этом заказчиков.

Альфа и бета тестирование тоже относятся к системным

Использование автотестов позволяет упростить и ускорить получение объективной информации о качестве продукта, особенно в agile практике!

Как писать тесты

Типичный составные части теста:

- 1.Инициализация начального состояния (set up)
- 2.Выполнение сценария (test)
- 3.Верификация конечного состояния (assert)
- 4.Очистка (tear down)

Шаги set up и tear down могут быть объединены для группы тестов.

```

import unittest
from src.odt.ODTParser import ODTParser
from src.odt.elements.ODTDocument import ODTDocument

class TestODTDefaultStyle(unittest.TestCase):
    def setUp(self) -> None:
        self.doc_path = "/Users/vladtereshch/PycharmProjects/normcontrol-Document-Parser/src/odt/documents/dipbac.odt"
        self.doc = ODTDocument(self.doc_path)
        self.odt_parser = ODTParser(self.doc)

    def test_get_default_styles(self):
        regular_styles = self.odt_parser.default_style_parser.get_default_styles(self.doc)
        list_of_styles = list(regular_styles)
        self.assertEqual(len(regular_styles), 6)
        self.assertEqual(regular_styles[list_of_styles[0]], {'family': 'table', 'table-properties/align': 'left',
                                                             'table-properties/border-model': 'collapsing',
                                                             'table-properties/margin-left': '0in',
                                                             'table-properties/writing-mode': 'lr-tb'})
        self.assertEqual(regular_styles[list_of_styles[5]], {'family': 'graphic', 'graphic-properties/fill': 'solid',
                                                             'graphic-properties/fill-color': '#4f81bd',
                                                             'graphic-properties/opacity': '100%',
                                                             'graphic-properties/stroke': 'solid',
                                                             'graphic-properties/stroke-width': '0.02778in',
                                                             'graphic-properties/stroke-color': '#385d8a',
                                                             'graphic-properties/stroke-opacity': '100%',
                                                             'graphic-properties/stroke-linecap': 'butt'})

    def test_get_default_style_by_family(self):
        regular_styles = self.odt_parser.default_style_parser.get_default_style_by_family(self.doc, 'graphic')
        self.assertEqual(regular_styles, {'family': 'graphic', 'graphic-properties/fill': 'solid',
                                           'graphic-properties/fill-color': '#4f81bd',
                                           'graphic-properties/opacity': '100%', 'graphic-properties/stroke': 'solid',
                                           'graphic-properties/stroke-width': '0.02778in',
                                           'graphic-properties/stroke-color': '#385d8a',
                                           'graphic-properties/stroke-opacity': '100%',
                                           'graphic-properties/stroke-linecap': 'butt'})

    def test_get_default_style_object_by_family(self):
        regular_styles = self.odt_parser.default_style_parser.get_default_style_object_by_family(self.doc, 'graphic')
        self.assertEqual(len(regular_styles.allowed_children), 11)

```

**Тесты супер важны, главное
не забывать их писать!**